

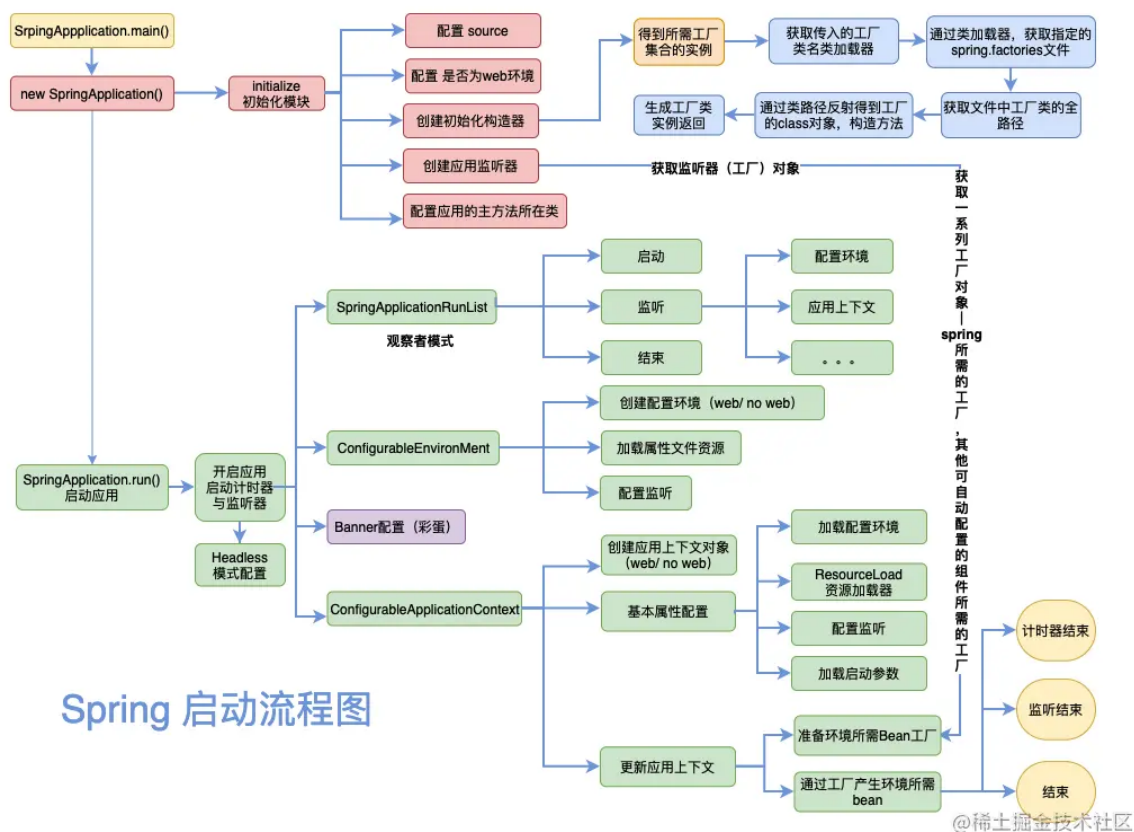
SpringBoot启动流程

一、前言

1.1 参考文章

- <https://juejin.cn/post/7035910505810100255>
- <https://www.cnblogs.com/huigui-mint/p/17517759.html>
- <https://cloud.tencent.com/developer/article/1874814>

1.2 启动流程图



1.3 遗留问题

- `EventPublishingRunListener` 中的成员变量 `SimpleApplicationEventMulticaster`、
- Spring启动过程：<https://www.cnblogs.com/summerday152/p/13639896.html#spring%E5%AEB9%E5%99%A8%E7%9A%84%E5%90%AF%E5%8A%A8%E5%85%A8%E6%B5%81%E7%A8%8B>

二、具体

2.1 核心内容

在SpringBoot的启动类中的内容，主要可以分为自动配置和启动流程两大部分。

1. 自动配置（总结△）

通过@SpringBootApplication注解来完成自动配置文件的引入，加载

2. 启动流程（总结△）

- java程序由启动主类调用main()方法开始。
- 调用 SpringApplication的构造方法，实例一个Spring应用对象。在构造方法里主要完成启动环境初始化工作，如，推断主类，spring应用类型，加载配置文件，读取spring.factories文件等。
- 调用run方法，所有的启动工作在该方法内完成，主要完成加载配置资源，准备上下文，创建上下文，刷新上下文，过程事件发布等。

2.2 自动配置

2.3 启动流程

SpringBoot的启动main方法如下示例所示

```
public static void main(String[] args) {  
    SpringApplication.run(DemoApplication.class, args);  
}
```

首先调用SpringApplication的构造器，然后再调用SpringApplication实例对象的run方法。

1. 构造器SpringApplication

SpringApplication构造器主要是初始化各种后期需要使用到类的实例（此时还没有IOC容器）

SpringApplication的构造器如下所示，将我们主程序的类对象作为传入，调用

```
public static ConfigurableApplicationContext run(Class<?>[] primarySources,  
string[] args) {  
    return new SpringApplication(primarySources).run(args);  
}
```

构造器的进一步调用

```
public SpringApplication(Class<?>... primarySources) {  
    this(null, primarySources);  
}
```

最终调用的构造器方法如下所示

```

/**
 * 创建一个新的SpringApplication实例。应用程序上下文将从指定的主源加载bean(有关详细信息, 请
 参阅类级文档)。实例可以在调用run(String...)之前自定义。
 * Create a new {@link SpringApplication} instance. The application context will
  load
 * beans from the specified primary sources (see {@link SpringApplication class-
  level}
 * documentation for details). The instance can be customized before calling
 * {@link #run(String...)}.
 * @param resourceLoader the resource loader to use
 * @param primarySources the primary bean sources
 * @see #run(Class, String[])
 * @see #setSources(Set)
 */
@SuppressWarnings({ "unchecked", "rawtypes" })
public SpringApplication(ResourceLoader resourceLoader, Class<?>...
primarySources) {

    // 将当前对象的resourceLoader属性设置为入参resourceLoader = null (为了在别的方法中能
    够使用, 提升变量作用域)
    this.resourceLoader = resourceLoader;
    // 断言primarySources不能为空
    Assert.notNull(primarySources, "PrimarySources must not be null");
    // 配置当前对象primarySources为入参primarySources, 由可变参数变成集合(格式转换)
    this.primarySources = new LinkedHashSet<>(Arrays.asList(primarySources));
    // 1. 判断当前程序类型
    this.webApplicationType = webApplicationType.deduceFromClasspath();
    // 2. 初始化BootstrapRegistryInitializer实例对象
    this.bootstrapRegistryInitializers = new ArrayList<>
(getSpringFactoriesInstances(BootstrapRegistryInitializer.class));
    // 3. 使用SpringFactoriesLoader 实例化所有可用的初始器
    setInitializers((Collection)
getSpringFactoriesInstances(ApplicationContextInitializer.class));
    // 4. 使用SpringFactoriesLoader 实例化所有可用的监听器
    setListeners((Collection)
getSpringFactoriesInstances(ApplicationListener.class));
    // 5. 配置应用主方法所在类
    this.mainApplicationClass = deduceMainApplicationClass();
}

```

1.配置应用主方法所在类

```
this.webApplicationType = webApplicationType.deduceFromClasspath();
```

主要根据classpath里面(通过调用ClassUtils.isPresent来判断)是否存在某个特征类(org.springframework.web.context.ConfigurableWebApplicationContext)来决定是否应该创建一个为Web应用使用的ApplicationContext类型。

```

public enum webApplicationType {

    /**
     * The application should not run as a web application and should not start
    an
     * embedded web server.

```

```

    */
    NONE,

    /**
     * The application should run as a servlet-based web application and should
    start an
     * embedded servlet web server.
     */
    SERVLET,

    /**
     * The application should run as a reactive web application and should start
    an
     * embedded reactive web server.
     */
    REACTIVE;

    private static final String[] SERVLET_INDICATOR_CLASSES = {
"javax.servlet.Servlet",
        "org.springframework.web.context.ConfigurableWebApplicationContext"
    };

    private static final String WEBMVC_INDICATOR_CLASS =
"org.springframework.web.servlet.DispatcherServlet";

    private static final String WEBFLUX_INDICATOR_CLASS =
"org.springframework.web.reactive.DispatcherHandler";

    private static final String JERSEY_INDICATOR_CLASS =
"org.glassfish.jersey.servlet.ServletContainer";

    static WebApplicationType deduceFromClasspath() {
        if (ClassUtils.isPresent(WEBFLUX_INDICATOR_CLASS, null) &&
!ClassUtils.isPresent(WEBMVC_INDICATOR_CLASS, null)
            && !ClassUtils.isPresent(JERSEY_INDICATOR_CLASS, null)) {
            return WebApplicationType.REACTIVE;
        }
        for (String className : SERVLET_INDICATOR_CLASSES) {
            if (!ClassUtils.isPresent(className, null)) {
                return WebApplicationType.NONE;
            }
        }
        return WebApplicationType.SERVLET;
    }
}

```

其查找类是否存在，是通过工具类ClassUtils的isPresent方法来实现，其调用forName将给定的字符串加载成对应的类

- 加载成功返回true
- 加载失败返回false

```

public static boolean isPresent(String className, @Nullable ClassLoader
classLoader) {

```

```

    try {
        forName(className, classLoader);
        return true;
    }
    catch (IllegalAccessException err) {
        throw new IllegalStateException("Readability mismatch in inheritance
        hierarchy of class [" +
            className + "]: " + err.getMessage(), err);
    }
    catch (Throwable ex) {
        // Typically ClassNotFoundException or NoClassDefFoundError...
        return false;
    }
}

```

2.初始化BootstrapRegistryInitializer实例对象

作用：获取系统配置引导类，BootstrapRegistry是SpringBoot启动的基础配置（系统配置引导类，比加载spring的bean更加初级，更加早）关于BootstrapRegistryInitializer的作用参考[文章](#)。

获取SpringBoot引导类，并将其转化为数组结构，设置给SpringApplication对象中

bootstrapRegistryInitializers属性

```

this.bootstrapRegistryInitializers = new ArrayList<>(
    getSpringFactoriesInstances(BootstrapRegistryInitializer.class));

```

其调用SpringApplication的方法来进行实例化

```

private <T> Collection<T> getSpringFactoriesInstances(Class<T> type, Class<?>[]
parameterTypes, Object... args) {
    // 获取类加载器
    ClassLoader classLoader = getClassLoader();
    // Use names and ensure unique to protect against duplicates
    // 获取要加载类的名称
    Set<String> names = new LinkedHashSet<>(
        SpringFactoriesLoader.loadFactoryNames(type, classLoader));
    // 创建实例
    List<T> instances = createSpringFactoriesInstances(type, parameterTypes,
        classLoader, args, names);
    // 设置加载的顺序
    AnnotationAwareOrderComparator.sort(instances);
    return instances;
}

```

- **获取类加载器**：使用 `getClassLoader()` 方法获取当前的类加载器。
- **加载工厂名称**：通过 `SpringFactoriesLoader.loadFactoryNames(type, classLoader)` 获取与给定类型相关的工厂名称，并使用 `LinkedHashSet` 确保唯一性以防止重复。
- **创建实例**：调用 `createSpringFactoriesInstances` 方法，使用加载的名称和提供的参数类型、类加载器以及构造参数来创建实际的实例。

- **排序实例：**使用 `AnnotationAwareOrderComparator.sort(instances)` 对创建的实例进行排序，以确保按优先级顺序排列。

首先获得类加载器，如果不存在，就获取默认类加载器

```
public ClassLoader getClassLoader() {  
    if (this.resourceLoader != null) {  
        return this.resourceLoader.getClassLoader();  
    }  
    return ClassUtils.getDefaultClassLoader();  
}
```

`SpringFactoriesLoader.loadFactoryNames` 获取要加载实例的类名称

```
public static List<String> loadFactoryNames(Class<?> factoryType, @Nullable  
ClassLoader classLoader) {  
    ClassLoader classLoaderToUse = classLoader;  
    if (classLoaderToUse == null) {  
        classLoaderToUse = SpringFactoriesLoader.class.getClassLoader();  
    }  
    String factoryTypeName = factoryType.getName();  
    return  
loadSpringFactories(classLoaderToUse).getOrDefault(factoryTypeName,  
Collections.emptyList());  
}
```

本质就是通过 `loadSpringFactories` 方法去各个jar包中的 `META-INF/spring.factories` 去获取

参考路径：`file:/D:/tool/develop/apache-maven-`

`3.8.8/mvn_reop/org/springframework/boot/spring-boot/2.7.10/spring-boot-`

`2.7.10.jar!/META-INF/spring.factories`

```
private static Map<String, List<String>> loadSpringFactories(ClassLoader  
classLoader) {  
    Map<String, List<String>> result = cache.get(classLoader);  
    if (result != null) {  
        return result;  
    }  
  
    result = new HashMap<>();  
    try {  
        Enumeration<URL> urls =  
classLoader.getResources(FACTORIES_RESOURCE_LOCATION);  
        while (urls.hasMoreElements()) {  
            URL url = urls.nextElement();  
            UrlResource resource = new UrlResource(url);  
            Properties properties =  
PropertiesLoaderUtils.loadProperties(resource);  
            for (Map.Entry<?, ?> entry : properties.entrySet()) {  
                String factoryTypeName = ((String) entry.getKey()).trim();  
                String[] factoryImplementationNames =
```

```

        StringUtils.commaDelimitedListToStringArray((String)
entry.getValue());
        for (String factoryImplementationName :
factoryImplementationNames) {
            result.computeIfAbsent(factoryTypeName, key -> new
ArrayList<>())
                .add(factoryImplementationName.trim());
        }
    }
}

// Replace all lists with unmodifiable lists containing unique
elements
result.replaceAll((factoryType, implementations) ->
implementations.stream().distinct()
    .collect(Collectors.collectingAndThen(Collectors.toList(),
Collections::unmodifiableList)));
cache.put(classLoader, result);
}
catch (IOException ex) {
    throw new IllegalArgumentException("Unable to load factories from
location [" +
        FACTORIES_RESOURCE_LOCATION + "]", ex);
}
return result;
}

```

createSpringFactoriesInstances创建实例方法

在引入的jar包 org.springframework.boot.spring-boot-autoconfigure 中的 META-INF 下的 spring.factories 中查找配置信息全路径名称为入参 parameterTypes 的，即 BootstrapRegistryInitializer.class 的类并加载（同样其他项目中的 META-INF 下的 spring.factories 中加了 BootstrapRegistryInitializer.class 对应的类配置也会被实例化）

```

private <T> List<T> createSpringFactoriesInstances(Class<T> type, Class<?>[]
parameterTypes,
    ClassLoader classLoader, Object[] args, Set<String> names) {
    List<T> instances = new ArrayList<>(names.size());
    for (String name : names) {
        try {
            Class<?> instanceClass = ClassUtils.forName(name, classLoader);
            Assert.isAssignable(type, instanceClass);
            Constructor<?> constructor =
instanceClass.getDeclaredConstructor(parameterTypes);
            T instance = (T) BeanUtils.instantiateClass(constructor, args);
            instances.add(instance);
        }
        catch (Throwable ex) {
            throw new IllegalArgumentException("Cannot instantiate " + type + " :
" + name, ex);
        }
    }
}

```

```

    }
    return instances;
}

```

- **初始化实例列表：**创建一个 `ArrayList`，大小为名称集合的大小，用于存储创建的实例。
- **遍历名称集合：**对 `names` 集合中的每个名称进行迭代。
- **加载类：**
 - 使用 `ClassUtils.forName(name, classLoader)` 根据名称和类加载器加载类。
 - 通过 `Assert.isAssignable(type, instanceClass)` 确保加载的类可以被赋值为目标类型 `T`。
- **获取构造函数：**
 - 通过 `instanceClass.getDeclaredConstructor(parameterTypes)` 获取匹配的构造函数。
- **实例化对象：**
 - 使用 `BeanUtils.instantiateClass(constructor, args)` 调用构造函数创建实例，并将其强制转换为类型 `T`。
- **添加到列表：**将创建的实例添加到 `instances` 列表中。
- **异常处理：**如果在创建过程中发生任何异常，抛出 `IllegalArgumentException`，并附上详细信息。

3.使用SpringFactoriesLoader 实例化所有可用的初始器

作用：获取 `ApplicationContextInitializer.class` 对应的实例

```

setInitializers((Collection)
getSpringFactoriesInstances(ApplicationContextInitializer.class));

```

首先调用 `getSpringFactoriesInstances` 方法获取所有的 `ApplicationContextInitializer` 对象

```

private <T> Collection<T> getSpringFactoriesInstances(Class<T> type) {
    return getSpringFactoriesInstances(type, new Class<?>[] {});
}

```

并将其值赋值给成员变量 `initializers`

```

public void setInitializers(Collection<? extends ApplicationContextInitializer<?>>
initializers) {
    this.initializers = new ArrayList<>(initializers);
}

```

同理和3一样，去 `META-INF` 下的 `spring.factories` 中查找对应的类

4.使用SpringFactoriesLoader 实例化所有可用的监听器

作用：获取ApplicationListener.class对应的监听器实例，SpringBoot通过监听器的方式对初始化过程及运行过程进行干预。

实现：同理2、3

扩展：

SpringBoot启动是个很庞大的过程，没有做对应的接口去定义每一个过程，而是用了事件监听机制，类似回调机制的方式去进行后期扩展，对需要干预的部分，定义相关的监听器，并且以事件对象作为区分（泛型的设置）

首先我们创建一个自定义监听器

```
package com.example.listener;

import org.springframework.context.ApplicationEvent;
import org.springframework.context.ApplicationListener;

/**
 * @author banana
 * @create 2024-09-28 15:26
 */
public class MyListener implements ApplicationListener {
    @Override
    public void onApplicationEvent(ApplicationEvent event) {
        System.out.println("=== test ===");
    }
}
```

然后将其加入到 META-INF 下的 spring.factories 中

```
# Application Listeners
org.springframework.context.ApplicationListener=\
com.example.listener.MyListener
```

我们启动SpringBoot项目，可以看到其被多次调用（回调）

```

C:\Program Files\Java\jdk1.8.0_91\bin\java.exe" ...
Connected to the target VM, address: '127.0.0.1:59237', transport: 'socket'

=== test ===
=== test ===

  /\ / _--' _-- ( ) _-- _-- \\\ \
( ( )\__| ' _| ' _| ' _| \\\ \
\W _--| | | | | | | | ( | ) ) )
' |__| | _| | | | | \_ _| / / / /
=====|_|=====|_ _/_/_/_/_/

:: Spring Boot ::                (v2.7.10)

=== test ===
2024-09-28 15:28:11.961 INFO 11000 --- [           main] com.example.DemoApplication : Starting
using Java 1.8.0_91 on LAPTOP-UI3GHR9L with PID 11000 (D:\codesaver\Banana-project\Banana-own-projects\target
by 23220 in D:\codesaver\Banana-project\Banana-own-projects)
2024-09-28 15:28:11.964 INFO 11000 --- [           main] com.example.DemoApplication : No active
falling back to 1 default profile: "default"
=== test ===
2024-09-28 15:28:13.077 INFO 11000 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat ir
port(s): 6666 (http)
2024-09-28 15:28:13.086 INFO 11000 --- [           main] o.apache.catalina.core.StandardService : Starting

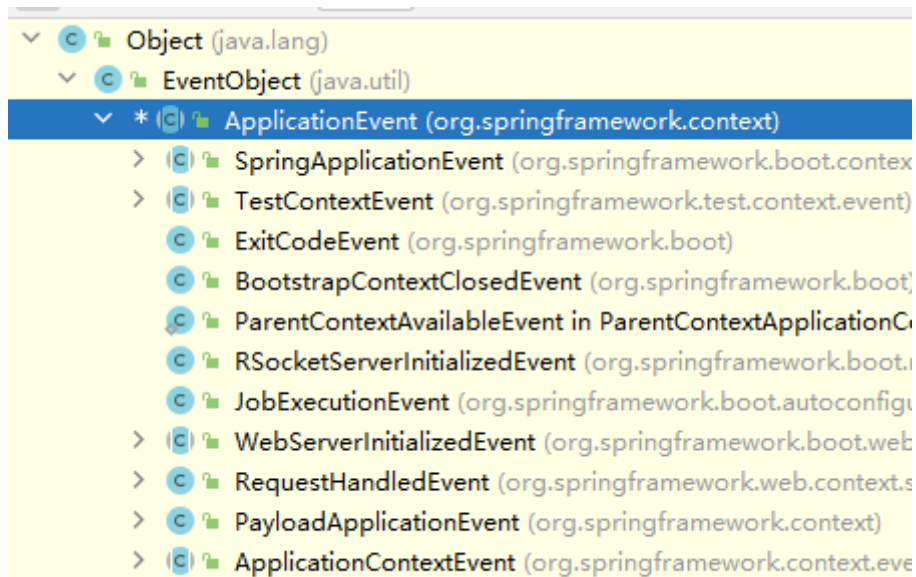
```

此时可以看到，由于没有指定事件，会在每一个监听事件的地方都执行，如果需要干预某一个具体的事件位置，通过泛型指定，如下所示

```
public class MyListener implements ApplicationListener<ApplicationStartedEvent> {

    @Override
    public void onApplicationEvent(ApplicationStartedEvent event) {
        System.out.println("=== test ===");
    }
}
```

看下ApplicationEvent的层次结构，可以看到有很多对应的事件，每一个事件对应整个过程中的某一个点



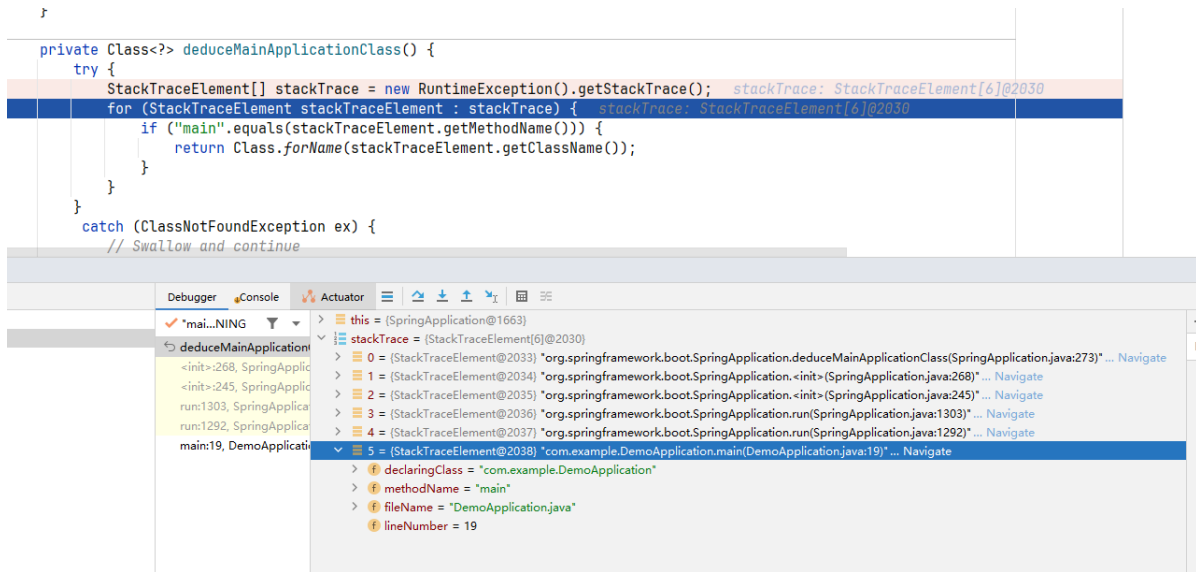
5.配置应用主方法所在类

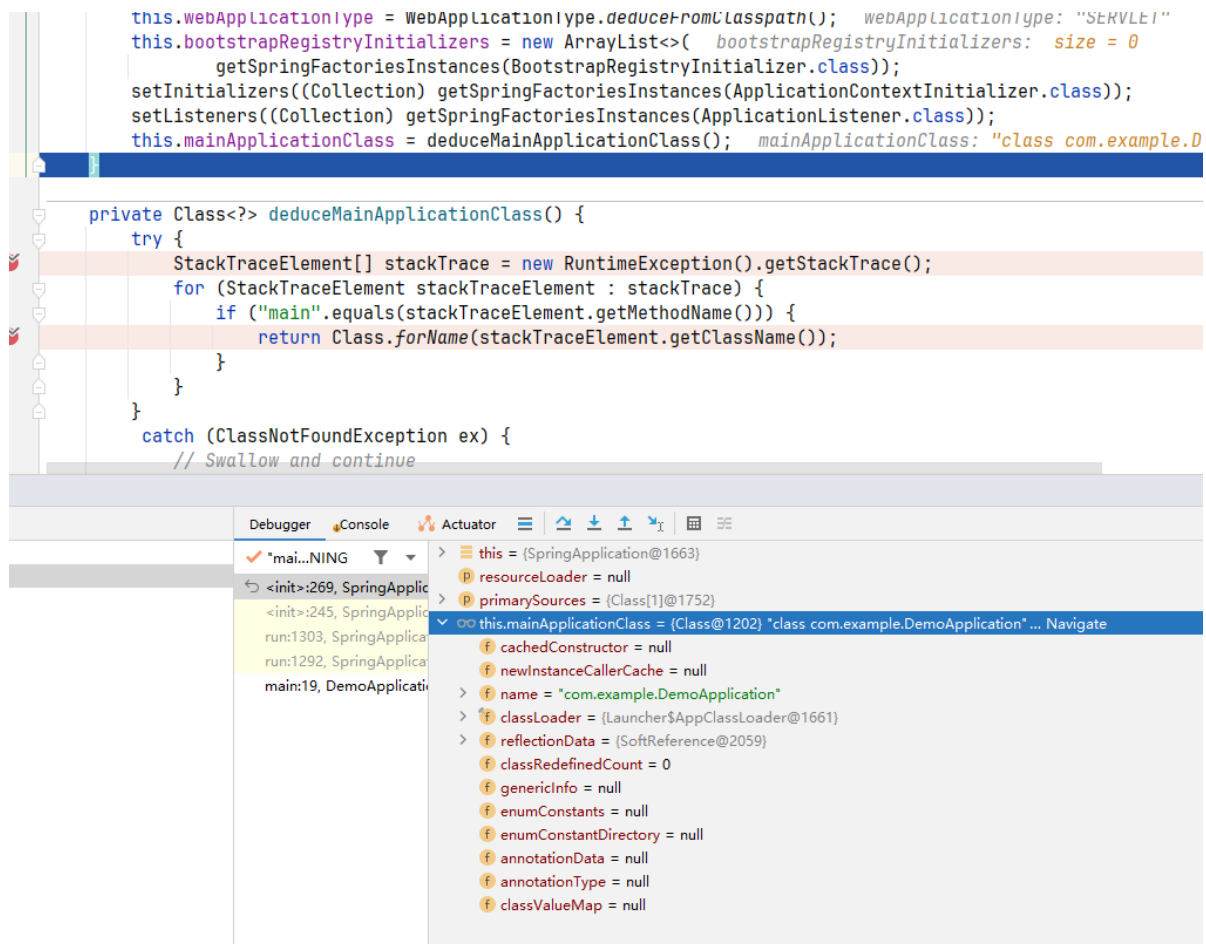
```
private Class<?> deduceMainApplicationClass() {
    try {
        StackTraceElement[] stackTrace = new RuntimeException().getStackTrace();
        for (StackTraceElement stackTraceElement : stackTrace) {
            if ("main".equals(stackTraceElement.getMethodName())) {
                return Class.forName(stackTraceElement.getClassName());
            }
        }
    }
    catch (ClassNotFoundException ex) {
        // Swallow and continue
    }
    return null;
}
```

主要功能是推断出主应用程序类（即包含 `main` 方法的类。

1. **获取堆栈跟踪**：通过创建一个 `RuntimeException` 实例并调用 `getStackTrace()`，获取当前线程的堆栈跟踪元素。

2. **遍历堆栈元素**: 循环检查每个堆栈元素, 查找方法名为 `main` 的元素。
3. **返回主类**: 如果找到 `main` 方法, 使用 `Class.forName()` 根据类名返回对应的 `Class` 对象。
4. **异常处理**: 如果在加载类时发生 `ClassNotFoundException`, 则捕获并忽略该异常, 继续执行。
5. **返回值**: 如果没有找到 `main` 方法, 或在类加载过程中出错, 最终返回 `null`。





2.启动方法RUN

整个过程就是在通过直接已经加载后的类去初始化容器，改方法调用后的结果就是获得一个IOC的上下文容器 `ApplicationContext`对象，具体实现类为 `ConfigurableApplicationContext`

```
public static void main(String[] args) {
    ConfigurableApplicationContext run =
    SpringApplication.run(DemoApplication.class, args);
}
```

其具体执行过程如下所示

```
/**
 * Run the Spring application, creating and refreshing a new
 * {@link ApplicationContext}.
 * @param args the application arguments (usually passed from a Java main method)
 * @return a running {@link ApplicationContext}
 */
public ConfigurableApplicationContext run(String... args) {
    // 计时：计时开始
    long startTime = System.nanoTime();
    // 0.创建系统引导信息对应的上下文对象（不等同于ApplicationContext）
    DefaultBootstrapContext bootstrapContext = createBootstrapContext();
    // 声明定义一个spring上下文对象
```

```

ConfigurableApplicationContext context = null;
// 1.模拟输入输出信号，避免出现因缺少外设导致的信号传输失败而引发错误（模拟显示器，键盘，鼠标）
// 让系统不存在io设备
configureHeadlessProperty();
// 2.获取当前所有可运行的监听器，并封装为listeners对象
SpringApplicationRunListeners listeners = getRunListeners(args);
// 3.监听器对象执行对应操作步骤（应用启动时）
listeners.starting(bootstrapContext, this.mainApplicationClass);
try {
    // 获取传入的参数值，并封装
    ApplicationArguments applicationArguments = new
DefaultApplicationArguments(args);
    // 4.将前期读取的数据加载为环境对象，用来描述信息
    ConfigurableEnvironment environment = prepareEnvironment(listeners,
bootstrapContext, applicationArguments);
    // 5.定义一个系统级参数
    configureIgnoreBeanInfo(environment);
    // 打印图标（初始化启动图标）
    Banner printedBanner = printBanner(environment);
    // 6.创建容器对象
    context = createApplicationContext();
    // 将一个 ApplicationStartup 实例设置到 ApplicationContext 中。这个实例通常用于
跟踪应用程序启动过程中的各种事件和性能指标。
    // Spring 提供了 ApplicationStartup 接口，可以用来收集启动过程中的相关信息，例如各
个环节的耗时等。这对于优化应用启动时间和性能分析非常有帮助
    // 在应用启动的不同阶段，可能会记录一些事件（如 Bean 的创建、上下文的初始化等），通过
ApplicationStartup，你可以获得更详细的启动过程日志
    // 开发者可以实现自定义的 ApplicationStartup 接口，从而定制如何收集和记录这些启动信
息

    context.setApplicationStartup(this.applicationStartup);
    // 7.对容器进行设置（相关参数来自前期的设定）
    prepareContext(bootstrapContext, context, environment, listeners,
applicationArguments, printedBanner);
    // 8.刷新容器环境
    refreshContext(context);
    // 刷新完毕后处理（空方法）
    afterRefresh(context, applicationArguments);

    // 计时：计时结束
    Duration timeTakenToStartup = Duration.ofNanos(System.nanoTime() -
startTime);
    // 是否启用日志记录启动信息
    if (this.logStartupInfo) {
        // 创建日志对应的对象，输出日志信息，保护启动的时间
        new
StartupInfoLogger(this.mainApplicationClass).logStarted(getApplicationLog(),
timeTakenToStartup);
    }
    // 3.监听器执行对应操作步骤
    listeners.started(context, timeTakenToStartup);
    // 10.△
    callRunners(context, applicationArguments);
}
catch (Throwable ex) {
    handleRunFailure(context, ex, listeners);
}

```

```

        throw new IllegalStateException(ex);
    }
    try {
        Duration timeTakenToReady = Duration.ofNanos(System.nanoTime() -
startTime);
        // 3. 监听器执行对应操作步骤
        listeners.ready(context, timeTakenToReady);
    }
    catch (Throwable ex) {
        handleRunFailure(context, ex, null);
        throw new IllegalStateException(ex);
    }
    return context;
}

```

0.创建系统引导信息对应的上下文对象（不等同于ApplicationContext）

```
DefaultBootstrapContext bootstrapContext = createBootstrapContext();
```

其调用SpringApplication的createBootstrapContext方法

```

private DefaultBootstrapContext createBootstrapContext() {
    // 创建一个引导上下文的默认实现，它通常用于存储启动时所需的配置信息和状态
    DefaultBootstrapContext bootstrapContext = new DefaultBootstrapContext();

    // bootstrapRegistryInitializers包含了多个注册的初始化器，这些初始化器负责在创建上下文
    后对其进行进一步配置或初始化
    // 遍历所有的 bootstrapRegistryInitializers。对于每个初始化器，调用其 initialize 方
    法，并将刚创建的 bootstrapContext 作为参数传递（猜测用于将初始化的一些信息维护到上下文中）
    this.bootstrapRegistryInitializers.forEach((initializer) ->
initializer.initialize(bootstrapContext));

    // 返回引导上下文对象
    return bootstrapContext;
}

```

1.模拟输入输出信号，避免出现因缺少外设导致的信号传输失败而引发错误（模拟显示器，键盘，鼠标）

本质是为系统参数添加 `java.awt.headless=true`，目的是做设备的兼容，避免在服务器上出现因缺少外设导致的信号传输失败而引发错误（模拟显示器，键盘，鼠标）

```

private static final String SYSTEM_PROPERTY_JAVA_AWT_HEADLESS =
"java.awt.headless";

private void configureHeadlessProperty() {
    System.setProperty(SYSTEM_PROPERTY_JAVA_AWT_HEADLESS,
        System.getProperty(SYSTEM_PROPERTY_JAVA_AWT_HEADLESS,
Boolean.toString(this.headless)));
}

```

可以通过如下方式获取所有的系统参数

```
Properties properties = System.getProperties();
properties.list(System.out)
```

2.获取当前所有可运行的监听器，并封装为listeners对象

```
SpringApplicationRunListeners listeners = getRunListeners(args);
```

其调用SpringApplication下的getRunListeners方法

```
private SpringApplicationRunListeners getRunListeners(String[] args) {
    // 获取SpringApplication和String的类对象
    Class<?>[] types = new Class<?>[] { SpringApplication.class, String[].class };
    // 调用SpringApplicationRunListeners构造器设置属性
    // 调用getSpringFactoriesInstances获取所有监听器
    return new SpringApplicationRunListeners(logger,
        getSpringFactoriesInstances(SpringApplicationRunListener.class,
            types, this, args),
        this.applicationStartup);
}
```

SpringApplicationRunListeners构造器

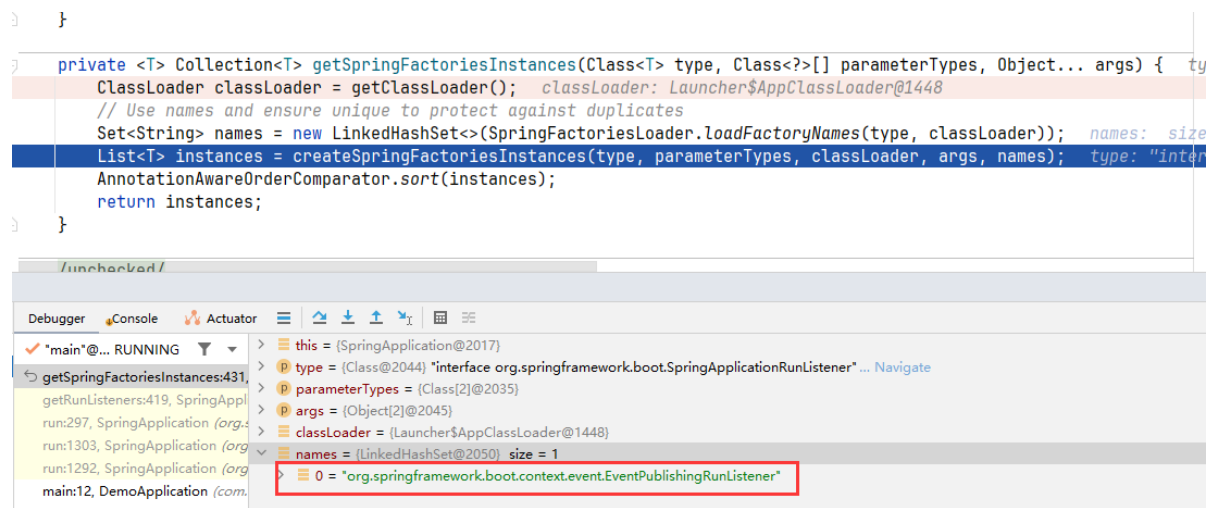
传递进来的参数设置给SpringApplicationRunListeners对象中的属性

```
SpringApplicationRunListeners(Log log, Collection<? extends
SpringApplicationRunListener> listeners,
    ApplicationStartup applicationStartup) {
    this.log = log;
    this.listeners = new ArrayList<>(listeners);
    this.applicationStartup = applicationStartup;
}
```

getSpringFactoriesInstances方法和之前的一样，就是去获取META-INF/spring.factories中配置的监听器全类名，然后根据反射的方式，通过构造器创造其实例

```
private <T> Collection<T> getSpringFactoriesInstances(Class<T> type, Class<?>[]
parameterTypes, Object... args) {
    ClassLoader classLoader = getClassLoader();
    // Use names and ensure unique to protect against duplicates
    Set<String> names = new LinkedHashSet<>
(SpringFactoriesLoader.loadFactoryNames(type, classLoader));
    List<T> instances = createSpringFactoriesInstances(type, parameterTypes,
classLoader, args, names);
    AnnotationAwareOrderComparator.sort(instances);
    return instances;
}
```

默认情况下该实例只有org.springframework.boot.context.event.EventPublishingRunListener



EventPublishingRunListener监听器负责在应用程序启动的不同阶段发布事件，作为springboot 的一个广播器，其具体源码如下所示

```
// 实现接口SpringApplicationRunListener 和 Ordered  
// SpringApplicationRunListener: 用于在应用程序的生命周期内提供回调  
// Ordered: 允许按顺序控制多个监听器的执行。  
public class EventPublishingRunListener implements SpringApplicationRunListener,  
Ordered {  
  
    private final SpringApplication application;  
  
    private final String[] args;  
  
    // SimpleApplicationEventMulticaster类  
    private final SimpleApplicationEventMulticaster initialMulticaster;  
  
    // EventPublishingRunListener构造器  
    // 初始化 application 和 args, 并将所有注册的 ApplicationListener 添加到  
    // initialMulticaster 中, 以便在后续阶段中可以发布事件  
    public EventPublishingRunListener(SpringApplication application, String[]  
args) {  
        this.application = application;  
        this.args = args;  
        this.initialMulticaster = new SimpleApplicationEventMulticaster();  
        for (ApplicationListener<?> listener : application.getListeners()) {  
            this.initialMulticaster.addApplicationListener(listener);  
        }  
    }  
  
    // 返回监听器的顺序, 值越小优先级越高。  
    @Override  
    public int getOrder() {  
        return 0;  
    }  
  
    // 在应用程序启动时发布 ApplicationStartingEvent。  
    @Override  
    public void starting(ConfigurableBootstrapContext bootstrapContext) {  
        this.initialMulticaster  
            .multicastEvent(new ApplicationStartingEvent(bootstrapContext,  
this.application, this.args));  
    }  
}
```



```

    }

    // 当环境准备好时，发布 ApplicationEnvironmentPreparedEvent
    @Override
    public void environmentPrepared(ConfigurableBootstrapContext
bootstrapContext,
        ConfigurableEnvironment environment) {
        this.initialMulticaster.multicastEvent(
            new ApplicationEnvironmentPreparedEvent(bootstrapContext,
this.application, this.args, environment));
    }

    // 当上下文准备好时，发布 ApplicationContextInitializedEvent。
    @Override
    public void contextPrepared(ConfigurableApplicationContext context) {
        this.initialMulticaster
            .multicastEvent(new
ApplicationContextInitializedEvent(this.application, this.args, context));
    }

    // 加载上下文时，将所有监听器添加到上下文，并发布 ApplicationPreparedEvent
    @Override
    public void contextLoaded(ConfigurableApplicationContext context) {
        for (ApplicationListener<?> listener : this.application.getListeners()) {
            if (listener instanceof ApplicationContextAware) {
                ((ApplicationContextAware)
listener).setApplicationContext(context);
            }
            context.addApplicationListener(listener);
        }
        this.initialMulticaster.multicastEvent(new
ApplicationPreparedEvent(this.application, this.args, context));
    }

    // 应用程序启动后，发布 ApplicationStartedEvent 和更新可用性状态
    @Override
    public void started(ConfigurableApplicationContext context, Duration
timeTaken) {
        context.publishEvent(new ApplicationStartedEvent(this.application,
this.args, context, timeTaken));
        AvailabilityChangeEvent.publish(context, LivenessState.CORRECT);
    }

    // 应用程序准备就绪时，发布 ApplicationReadyEvent
    @Override
    public void ready(ConfigurableApplicationContext context, Duration timeTaken)
    {
        context.publishEvent(new ApplicationReadyEvent(this.application,
this.args, context, timeTaken));
        AvailabilityChangeEvent.publish(context,
ReadinessState.ACCEPTING_TRAFFIC);
    }

    // 当应用程序启动失败时，发布 ApplicationFailedEvent。如果上下文活跃，则通过上下文发布
    事件；否则，通过初始的多播器发布
    @Override

```

```

    public void failed(ConfigurableApplicationContext context, Throwable
exception) {
        ApplicationFailedEvent event = new
ApplicationFailedEvent(this.application, this.args, context, exception);
        if (context != null && context.isActive()) {
            // Listeners have been registered to the application context so we
should
            // use it at this point if we can
            context.publishEvent(event);
        }
        else {
            // An inactive context may not have a multicaster so we use our
multicaster to
            // call all the context's listeners instead
            if (context instanceof AbstractApplicationContext) {
                for (ApplicationListener<?> listener :
((AbstractApplicationContext) context)
                    .getApplicationListeners()) {
                    this.initialMulticaster.addApplicationListener(listener);
                }
            }
            this.initialMulticaster.setErrorHandler(new LoggingErrorHandler());
            this.initialMulticaster.multicastEvent(event);
        }
    }
}

// LoggingErrorHandler: 实现了 ErrorHandler 接口, 用于处理事件监听器调用中的错误, 记
录警告日志
private static class LoggingErrorHandler implements ErrorHandler {

    private static final Log logger =
LogFactory.getLog(EventPublishingRunListener.class);

    @Override
    public void handleError(Throwable throwable) {
        logger.warn("Error calling ApplicationEventListener", throwable);
    }
}
}

```

补充:

1、关于 `SpringApplicationRunListener` 和 `ApplicationListener` 监听器

SpringApplicationRunListener

- **目的:** 此接口旨在支持 Spring Boot 应用程序的启动过程。它允许开发者在应用程序启动的不同阶段进行回调。
- **使用场景:** 用于在应用生命周期的特定点（如启动、环境准备、上下文加载等）执行逻辑。

- **实现方式:** 开发者可以实现该接口来定义在应用程序运行期间需要执行的操作，比如初始化设置、发布事件等。
- **主要方法:**
 - `starting()`
 - `environmentPrepared()`
 - `contextPrepared()`
 - `contextLoaded()`
 - `started()`
 - `ready()`
 - `failed()`

ApplicationListener

- **目的:** 此接口是用于处理 Spring 应用中的事件。通过实现此接口，开发者可以监听特定的事件并执行相应的逻辑。
- **使用场景:** 用于响应应用程序中的事件，例如用户自定义事件或 Spring 提供的标准事件（如上下文刷新、bean 创建等）。
- **实现方式:** 开发者实现该接口并重写 `onApplicationEvent(T event)` 方法，来定义对特定事件的响应行为。
- **主要方法:**
 - `onApplicationEvent(T event)`

除上述外，他们的加载位置也是不同的，可以参考文章：https://blog.csdn.net/weixin_51110958/article/details/123900949

3.监听器对象执行对应操作步骤（应用启动时）

```
listeners.starting(bootstrapContext, this.mainApplicationClass);
```

其调用SpringApplicationRunListeners对象的starting方法，传入两个参数

- bootstrapContext：系统引导信息对应的上下文对象
- mainApplicationClass：主函数的类对象

该方法在应用程序启动时被调用，目的是通知所有注册的 `SpringApplicationRunListener` 监听器，应用正在启动，调用 `dowithListeners()` 方法，将启动上下文传递给所有注册的监听器。

```

void starting(ConfigurableBootstrapContext bootstrapContext, Class<?>
mainApplicationClass) {
    doWithListeners("spring.boot.application.starting", (listener) ->
listener.starting(bootstrapContext),
        (step) -> {
            if (mainApplicationClass != null) {
                step.tag("mainApplicationClass",
mainApplicationClass.getName());
            }
        });
}

```

其中又调用SpringApplicationRunListeners对象的doWithListeners方法，传入三个参数

- `stepName`: 启动步骤的名称，用于标识当前的操作。
- `listenerAction`: 对每个监听器执行的操作（如调用 `starting()` 方法）。
- `stepAction`: 可选的附加操作，用于进一步处理 `StartupStep`。

```

private void doWithListeners(String stepName,
Consumer<SpringApplicationRunListener> listenerAction, Consumer<StartupStep>
stepAction) {
    // 创建一个新的启动步骤 (StartupStep) 并开始计时
    StartupStep step = this.applicationStartup.start(stepName);
    // 遍历所有的 SpringApplicationRunListener 监听器并对其执行 listenerAction (在这里
    是调用 starting() 方法)
    this.listeners.forEach(listenerAction);
    // 如果提供了 stepAction, 则执行它 (例如, 添加标签)
    if (stepAction != null) {
        stepAction.accept(step);
    }
    // 结束该启动步骤, 记录耗时等信息
    step.end();
}

```

4.将前期读取的数据加载为环境对象，用来描述信息

```

ConfigurableEnvironment environment = prepareEnvironment(listeners,
bootstrapContext, applicationArguments);

```

调用SpringApplication的prepareEnvironment方法

```

private ConfigurableEnvironment prepareEnvironment(SpringApplicationRunListeners
listeners,
    DefaultBootstrapContext bootstrapContext, ApplicationArguments
applicationArguments) {
    // Create and configure the environment
    // 根据不同环境不同的Environment (StandardServletEnvironment,
StandardReactiveWebEnvironment, StandardEnvironment)
    ConfigurableEnvironment environment = getOrCreateEnvironment();
    // 填充启动类参数到environment 对象
    configureEnvironment(environment, applicationArguments.getSourceArgs());
}

```

```

// 更新参数
ConfigurationPropertySources.attach(environment);
// 发布事件
listeners.environmentPrepared(bootstrapContext, environment);
// 绑定主类
DefaultPropertiesPropertySource.moveToEnd(environment);
Assert.state(!environment.containsProperty("spring.main.environment-prefix"),
    "Environment prefix cannot be set via properties.");
bindToSpringApplication(environment);
// 转换environment的类型，但这里应该类型和deduce的相同不用转换
if (!this.isCustomEnvironment) {
    EnvironmentConverter environmentConverter = new
EnvironmentConverter(getClassLoader());
    environment =
environmentConverter.convertEnvironmentIfNecessary(environment,
deduceEnvironmentClass());
}
// 将现有参数有封装成propertySources
ConfigurationPropertySources.attach(environment);
return environment;
}

```

5.定义一个系统级参数

```
configureIgnoreBeanInfo(environment);
```

其中方法内容如下所示

```

private void configureIgnoreBeanInfo(ConfigurableEnvironment environment) {
    // 首先检查系统属性中是否已经设置了 IGNORE_BEANINFO_PROPERTY_NAME
    if
(System.getProperty(CachedIntrospectionResults.IGNORE_BEANINFO_PROPERTY_NAME) ==
null) {
        // 从 ConfigurableEnvironment 中获取名为 IGNORE_BEANINFO_PROPERTY_NAME 的属
        性值。如果该属性不存在，则使用默认值 Boolean.TRUE
        Boolean ignore = environment.getProperty(
            CachedIntrospectionResults.IGNORE_BEANINFO_PROPERTY_NAME,
            Boolean.class,
            Boolean.TRUE
        );
        // 将获取到的布尔值（转为字符串）设置为系统属性。这意味着后续的调用将能够访问这个属性的
        值

        System.setProperty(CachedIntrospectionResults.IGNORE_BEANINFO_PROPERTY_NAME,
            ignore.toString());
    }
}

```

作用：

- Bean 信息缓存:

- `CachedIntrospectionResults` 是 Spring 用于缓存 Bean 的反射信息的类。通过设置 `IGNORE_BEANINFO_PROPERTY_NAME` 属性，可以控制是否忽略 Bean 信息的缓存。
- 提高性能:
 - 在某些情况下，忽略 Bean 信息的缓存可能会提高性能，特别是在动态创建或修改 Bean 时。
- 避免重复设置:
 - 通过检查系统属性是否已经设置，避免多次重复配置，提高效率。

6.创建容器对象

```
context = createApplicationContext();
```

根据当前应用类型创建不同的容器

```
protected ConfigurableApplicationContext createApplicationContext() {
    return this.applicationContextFactory.create(this.webApplicationType);
}
```

7.对容器进行设置（相关参数来自前期的设定）

```
prepareContext(bootstrapContext, context, environment, listeners,
    applicationArguments, printedBanner);
```

其调用Application的prepareContext方法

```
private void prepareContext(DefaultBootstrapContext bootstrapContext,
    ConfigurableApplicationContext context,
    ConfigurableEnvironment environment, SpringApplicationRunListeners
    listeners,
    ApplicationArguments applicationArguments, Banner printedBanner) {
    // 上下文绑定环境
    context.setEnvironment(environment);
    // 如果application有设置beanNameGenerator、resourceLoader就将其注入到上下文中，并将
    转换工具也注入到上下文中
    // Bean名称生成器:beanNameGenerator （自定义的 Bean 名称生成器可以影响 Bean 的创建过
    程，确保 Bean 的命名符合特定的规则）
    // 资源加载器: resourceLoader （）
    // addConversionService = true，从环境中获取转换服务并将其设置到 Bean 工厂中， 转换服
    务用于支持类型转换和属性绑定，是 Spring 中数据绑定和类型转换的重要组成部分，确保在不同类型之间进
    行有效转换
    postProcessApplicationContext(context);
    // 调用初始化的切面
    applyInitializers(context);
    // 发布ApplicationContextInitializedEvent事件
    listeners.contextPrepared(context);
    // 关闭上下文 ConfigurableApplicationContext 并发布一个事件，表示上下文已被关闭
    bootstrapContext.close(context);
    // 日志
    if (this.logStartupInfo) {
        logStartupInfo(context.getParent() == null);
    }
}
```

```

        logStartupProfileInfo(context);
    }
    // Add boot specific singleton beans
    // 获取当前上下文的 BeanFactory (IOC容器)
    ConfigurableListableBeanFactory beanFactory = context.getBeanFactory();
    // 将 applicationArguments 注册为单例 Bean, 名称为 springApplicationArguments。这
    使得在应用中可以通过依赖注入获取命令行参数
    beanFactory.registerSingleton("springApplicationArguments",
    applicationArguments);
    // 如果存在打印的横幅 (banner), 则将其注册为名为 springBootBanner 的单例 Bean。这样可
    以在应用中使用该 banner
    if (printedBanner != null) {
        beanFactory.registerSingleton("springBootBanner", printedBanner);
    }
    // 首先检查 beanFactory 是否是 AbstractAutowireCapableBeanFactory 的实例, 这样可以
    设置是否允许循环引用
    if (beanFactory instanceof AbstractAutowireCapableBeanFactory) {
        ((AbstractAutowireCapableBeanFactory)
    beanFactory).setAllowCircularReferences(this.allowCircularReferences);
        // 如果 beanFactory 是 DefaultListableBeanFactory, 则设置是否允许 Bean 定义覆
    盖。这两项配置影响 Bean 的创建和管理策略
        if (beanFactory instanceof DefaultListableBeanFactory) {
            ((DefaultListableBeanFactory) beanFactory)

.setAllowBeanDefinitionOverriding(this.allowBeanDefinitionOverriding);
        }
    }
    // 如果启用了懒初始化, 则添加一个 BeanFactoryPostProcessor后处理器, 使得 Bean 的初始化
    是延迟的, 直到它们被请求时才会被创建。这有助于提高启动速度和减少资源消耗
    if (this.lazyInitialization) {
        context.addBeanFactoryPostProcessor(new
    LazyInitializationBeanFactoryPostProcessor());
    }
    // 添加一个用于处理属性源排序的 BeanFactoryPostProcessor。这确保了在上下文中加载的属性
    源按照特定顺序处理, 以便正确解析配置
    context.addBeanFactoryPostProcessor(new
    PropertySourceOrderingBeanFactoryPostProcessor(context));

    // Load the sources
    // 获取所有应用的源 (例如配置类、XML 文件等)
    // 这里获取到的是BootstrapImportSelectorConfiguration这个class, 而不是自己写的启动
    来, 这个class是在之前注册的BootstrapApplicationListener的监听方法中注入的
    Set<Object> sources = getAllSources();
    // 确保源不为空
    Assert.notEmpty(sources, "Sources must not be empty");
    // 加载sources 到上下文中
    load(context, sources.toArray(new Object[0]));
    //发布ApplicationPreparedEvent事件
    listeners.contextLoaded(context);
}

```

postProcessApplicationContext方法如下所示

```

protected void postProcessApplicationContext(ConfigurableApplicationContext
context) {

```

```

        if (this.beanNameGenerator != null) {
            context.getBeanFactory()

            .registerSingleton(AnnotationConfigUtils.CONFIGURATION_BEAN_NAME_GENERATOR,
            this.beanNameGenerator);
        }
        if (this.resourceLoader != null) {
            if (context instanceof GenericApplicationContext) {
                ((GenericApplicationContext)
                context).setResourceLoader(this.resourceLoader);
            }
            if (context instanceof DefaultResourceLoader) {
                ((DefaultResourceLoader)
                context).setClassLoader(this.resourceLoader.getClassLoader());
            }
        }
        if (this.addConversionService) {

            context.getBeanFactory().setConversionService(context.getEnvironment().getConversionService());
        }
    }
}

```

applyInitializers(context)方法如下所示

```

protected void applyInitializers(ConfigurableApplicationContext context) {
    // 通过 getInitializers() 方法获取所有需要应用于上下文的
    ApplicationContextInitializer 实例。这些初始器通常用于在上下文刷新之前进行自定义配置
    for (ApplicationContextInitializer initializer : getInitializers()) {
        // 使用 GenericTypeResolver 来解析 initializer 的类型参数，确保它与
        ApplicationContextInitializer 的预期类型兼容（确保了在调用初始器时，传入的上下文类型是符合
        要求的）
        Class<?> requiredType =
        GenericTypeResolver.resolveTypeArgument(initializer.getClass(),
            ApplicationContextInitializer.class);
        Assert.isInstanceOf(requiredType, context, "Unable to call
        initializer.");
        // 调用每个初始器的 initialize 方法，传入当前的 context。这使得每个初始器可以对应用
        上下文进行配置或调整，例如添加属性源、设置环境等
        initializer.initialize(context);
    }
}

```

8.刷新容器环境

```
refreshContext(context);
```

调用refreshContext方法，传入需要刷新的应用上下文


```
private void refreshContext(ConfigurableApplicationContext context) {
    // 调用 shutdownHook.registerApplicationContext(context) 注册应用上下文。这通常用于在 JVM 关闭时优雅地关闭 Spring 应用程序，例如释放资源和销毁 Bean。
    if (this.registerShutdownHook) {
        // 对象通常是一个实现了关闭逻辑的类，负责管理应用程序的生命周期
        shutdownHook.registerApplicationContext(context);
    }
    // 刷新上下文。这通常涉及重新加载 Bean 定义、初始化 Bean 和重新配置应用上下文等操作 (Spring中的内容)
    refresh(context);
}
```

refresh方法

```
protected void refresh(ConfigurableApplicationContext applicationContext) {
    applicationContext.refresh();
}
```

Spring的刷新方法

```
public void refresh() throws BeansException, IllegalStateException {
    synchronized (this.startupShutdownMonitor) {
        //记录启动时间、状态，web容器初始化其property，复制listener
        prepareRefresh();
        //这里返回的是context的BeanFactory
        ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
        //beanFactory注入一些标准组件，例如ApplicationContextAwareProcessor，
        //ClassLoader等
        prepareBeanFactory(beanFactory);
        try {
            //给实现类留的一个钩子，例如注入BeanPostProcessors，这里是个空方法
            postProcessBeanFactory(beanFactory);

            // 调用切面方法
            invokeBeanFactoryPostProcessors(beanFactory);

            // 注册切面bean
            registerBeanPostProcessors(beanFactory);

            // Initialize message source for this context.
            initMessageSource();

            // bean工厂注册一个key为applicationEventMulticaster的广播器
            initApplicationEventMulticaster();

            // 给实现类留的一钩子，可以执行其他refresh的工作，这里是个空方法
            onRefresh();

            // 将listener注册到广播器中
            registerListeners();

            // 实例化未实例化的bean
            finishBeanFactoryInitialization(beanFactory);
```

```

        // 清理缓存, 注入DefaultLifecycleProcessor, 发布ContextRefreshedEvent
        finishRefresh();
    }

    catch (BeansException ex) {
        if (logger.isWarnEnabled()) {
            logger.warn("Exception encountered during context initialization
- " +
                        "cancelling refresh attempt: " + ex);
        }

        // Destroy already created singletons to avoid dangling resources.
        destroyBeans();

        // Reset 'active' flag.
        cancelRefresh(ex);

        // Propagate exception to caller.
        throw ex;
    }

    finally {
        // Reset common introspection caches in Spring's core, since we
        // might not ever need metadata for singleton beans anymore...
        resetCommonCaches();
    }
}
}
}

```

10.Δ

调用在 Spring 应用上下文中定义的 `ApplicationRunner` 和 `CommandLineRunner` 类型的 Bean。这些 Runner 通常用于在应用启动后执行一些特定的逻辑，比如初始化数据或执行某些任务

```
callRunners(context, applicationArguments);
```

调用callRunners方法详情

```

private void callRunners(ApplicationContext context, ApplicationArguments args) {
    // 创建一个列表 runners, 并将上下文中所有实现了 ApplicationRunner 和
    CommandLineRunner 接口的 Bean 添加到列表中。这两种 Runner 接口通常用于定义应用程序启动后的
    行为。
    List<Object> runners = new ArrayList<>();
    runners.addAll(context.getBeansOfType(ApplicationRunner.class).values());
    runners.addAll(context.getBeansOfType(CommandLineRunner.class).values());
    // 使用 AnnotationAwareOrderComparator 对 Runner 列表进行排序, 以确保按照指定的顺序
    执行。这个排序机制基于 Spring 的 @Order 注解或其他排序逻辑, 确保依赖关系得到满足。
    AnnotationAwareOrderComparator.sort(runners);
    // 遍历去重后的 Runner 列表 (使用 LinkedHashSet 确保唯一性和插入顺序), 并根据类型调用
    相应的 callRunner 方法
    for (Object runner : new LinkedHashSet<>(runners)) {

```

```

        if (runner instanceof ApplicationRunner) {
            callRunner((ApplicationRunner) runner, args);
        }
        if (runner instanceof CommandLineRunner) {
            callRunner((CommandLineRunner) runner, args);
        }
    }
}

```

三、相关内容

1、SpringBootApplication注解

该注解SpringBoot给予的说明

Indicates a configuration class that declares one or more @Bean methods and also triggers auto-configuration and component scanning. This is a convenience annotation that is equivalent to declaring @Configuration, @EnableAutoConfiguration and @ComponentScan.

指示一个配置类，它声明一个或多个@Bean方法，并触发自动配置和组件扫描。这是一个方便的注释，相当于声明@Configuration， @EnableAutoConfiguration和@ComponentScan。

@SpringBootApplication

本质是一个配置类，其内部除了@Target、@Retention、@Documented，@Inherited，由三个注解组成

```

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = { @Filter(type = FilterType.CUSTOM, classes =
TypeExcludeFilter.class),
    @Filter(type = FilterType.CUSTOM, classes =
AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication {

    /**
     * Exclude specific auto-configuration classes such that they will never be
     applied.
     * @return the classes to exclude
     */
    @AliasFor(annotation = EnableAutoConfiguration.class)
    Class<?>[] exclude() default {};

    /**

```

```

    * Exclude specific auto-configuration class names such that they will never
be
    * applied.
    * @return the class names to exclude
    * @since 1.3.0
    */
@AliasFor(annotation = EnableAutoConfiguration.class)
String[] excludeName() default {};

/**
    * Base packages to scan for annotated components. Use {@link
#scanBasePackageClasses}
    * for a type-safe alternative to String-based package names.
    * <p>
    * <strong>Note:</strong> this setting is an alias for
    * {@link ComponentScan @ComponentScan} only. It has no effect on {@code
@Entity}
    * scanning or Spring Data {@link Repository} scanning. For those you should
add
    * {@link org.springframework.boot.autoconfigure.domain.EntityScan
@EntityScan} and
    * {@code @Enable...Repositories} annotations.
    * @return base packages to scan
    * @since 1.3.0
    */
@AliasFor(annotation = ComponentScan.class, attribute = "basePackages")
String[] scanBasePackages() default {};

/**
    * Type-safe alternative to {@link #scanBasePackages} for specifying the
packages to
    * scan for annotated components. The package of each class specified will be
scanned.
    * <p>
    * Consider creating a special no-op marker class or interface in each
package that
    * serves no purpose other than being referenced by this attribute.
    * <p>
    * <strong>Note:</strong> this setting is an alias for
    * {@link ComponentScan @ComponentScan} only. It has no effect on {@code
@Entity}
    * scanning or Spring Data {@link Repository} scanning. For those you should
add
    * {@link org.springframework.boot.autoconfigure.domain.EntityScan
@EntityScan} and
    * {@code @Enable...Repositories} annotations.
    * @return base packages to scan
    * @since 1.3.0
    */
@AliasFor(annotation = ComponentScan.class, attribute = "basePackageClasses")
Class<?>[] scanBasePackageClasses() default {};

/**
    * The {@link BeanNameGenerator} class to be used for naming detected
components
    * within the Spring container.

```

```

    * <p>
    * The default value of the {@link BeanNameGenerator} interface itself
    indicates that
    * the scanner used to process this {@code @SpringBootApplication} annotation
    should
    * use its inherited bean name generator, e.g. the default
    * {@link AnnotationBeanNameGenerator} or any custom instance supplied to the
    * application context at bootstrap time.
    * @return {@link BeanNameGenerator} to use
    * @see SpringApplication#setBeanNameGenerator(BeanNameGenerator)
    * @since 2.3.0
    */
    @AliasFor(annotation = ComponentScan.class, attribute = "nameGenerator")
    Class<? extends BeanNameGenerator> nameGenerator() default
    BeanNameGenerator.class;

    /**
    * Specify whether {@link Bean @Bean} methods should get proxied in order to
    enforce
    * bean lifecycle behavior, e.g. to return shared singleton bean instances
    even in
    * case of direct {@code @Bean} method calls in user code. This feature
    requires
    * method interception, implemented through a runtime-generated CGLIB
    subclass which
    * comes with limitations such as the configuration class and its methods not
    being
    * allowed to declare {@code final}.
    * <p>
    * The default is {@code true}, allowing for 'inter-bean references' within
    the
    * configuration class as well as for external calls to this configuration's
    * {@code @Bean} methods, e.g. from another configuration class. If this is
    not needed
    * since each of this particular configuration's {@code @Bean} methods is
    * self-contained and designed as a plain factory method for container use,
    switch
    * this flag to {@code false} in order to avoid CGLIB subclass processing.
    * <p>
    * Turning off bean method interception effectively processes {@code @Bean}
    methods
    * individually like when declared on non-{@code @Configuration} classes,
    a.k.a.
    * "@Bean Lite Mode" (see {@link Bean @Bean's javadoc}). It is therefore
    behaviorally
    * equivalent to removing the {@code @Configuration} stereotype.
    * @since 2.2
    * @return whether to proxy {@code @Bean} methods
    */
    @AliasFor(annotation = Configuration.class)
    boolean proxyBeanMethods() default true;
}

```

属性说明（本质上吧很多注解都拿出来，以别名的方式在这里进行配置）

- exclude

排除特定的自动配置类，使它们永远不会被应用。

返回: 要排除的类

```
/**
 * Exclude specific auto-configuration classes such that they will never be
 * applied.
 * @return the classes to exclude
 */
@AliasFor(annotation = EnableAutoConfiguration.class)
Class<?>[] exclude() default {};
```

- excludeName

排除特定的自动配置类名，使它们永远不会被应用。

返回: 要排除的类名

```
/**
 * Exclude specific auto-configuration class names such that they will never be
 * applied.
 * @return the class names to exclude
 * @since 1.3.0
 */
@AliasFor(annotation = EnableAutoConfiguration.class)
String[] excludeName() default {};
```

其中@AliasFor注解

```
@AliasFor(annotation = EnableAutoConfiguration.class)
```

继承 @EnableAutoConfiguration 的excludeName属性，可以使用 @AliasFor 来直接映射这些属性

- scanBasePackages

扫描带注释的组件的基本包。使用scanbasepackageclass作为基于字符串的包名的类型安全替代方案。

注意: 这个设置只是@ComponentScan的别名。它对@Entity扫描或Spring Data Repository扫描没有影响。对于那些你应该添加@ entitscan和@Enable...存储库注释。

返回: 要扫描的基本包

```
/**
 * Base packages to scan for annotated components. Use {@link
 * #scanBasePackageClasses}
 * for a type-safe alternative to String-based package names.
 * <p>
 * <strong>Note:</strong> this setting is an alias for
 * {@link ComponentScan @ComponentScan} only. It has no effect on {@code @Entity}
 * scanning or Spring Data {@link Repository} scanning. For those you should add
```

```

* {@link org.springframework.boot.autoconfigure.domain.EntityScan @EntityScan}
and
* {@code @Enable...Repositories} annotations.
* @return base packages to scan
* @since 1.3.0
*/
@AliasFor(annotation = ComponentScan.class, attribute = "basePackages")
String[] scanBasePackages() default {};

```

其与 `@ComponentScan` 注解的 `basePackages` 属性的别名关系

```

@AliasFor(annotation = ComponentScan.class, attribute = "basePackages")

```

- `scanBasePackageClasses`

`scanBasePackages`的类型安全替代方案，用于指定要扫描的包以查找带注释的组件。指定的每个类的包将被扫描。

考虑在每个包中创建一个特殊的无操作标记类或接口，它除了被这个属性引用之外没有其他用途。

注意:这个设置只是`@ComponentScan`的别名。它对`@Entity`扫描或Spring Data Repository扫描没有影响。对于那些你应该添加`@EntityScan`和`@Enable...`存储库注释。

返回:

要扫描的基本包

```

/**
 * Type-safe alternative to {@link #scanBasePackages} for specifying the
 * packages to
 * scan for annotated components. The package of each class specified will be
 * scanned.
 * <p>
 * Consider creating a special no-op marker class or interface in each
 * package that
 * serves no purpose other than being referenced by this attribute.
 * <p>
 * <strong>Note:</strong> this setting is an alias for
 * {@link ComponentScan @ComponentScan} only. It has no effect on {@code
 * @Entity}
 * scanning or Spring Data {@link Repository} scanning. For those you should
 * add
 * {@link org.springframework.boot.autoconfigure.domain.EntityScan
 * @EntityScan} and
 * {@code @Enable...Repositories} annotations.
 * @return base packages to scan
 * @since 1.3.0
 */
@AliasFor(annotation = ComponentScan.class, attribute = "basePackageClasses")
Class<?>[] scanBasePackageClasses() default {};

```

- `nameGenerator`

BeanNameGenerator类用于命名Spring容器中检测到的组件。

BeanNameGenerator接口本身的默认值表明，用于处理@SpringBootApplication注释的扫描器应该使用其继承的bean名称生成器，例如，默认的AnnotationBeanNameGenerator或在引导时提供给应用程序上下文的任何自定义实例。

返回：

使用BeanNameGenerator

```
/**
 * The {@link BeanNameGenerator} class to be used for naming detected
 components
 * within the Spring container.
 * <p>
 * The default value of the {@link BeanNameGenerator} interface itself
 indicates that
 * the scanner used to process this {@code @SpringBootApplication} annotation
 should
 * use its inherited bean name generator, e.g. the default
 * {@link AnnotationBeanNameGenerator} or any custom instance supplied to the
 * application context at bootstrap time.
 * @return {@link BeanNameGenerator} to use
 * @see SpringApplication#setBeanNameGenerator(BeanNameGenerator)
 * @since 2.3.0
 */
@AliasFor(annotation = ComponentScan.class, attribute = "nameGenerator")
Class<? extends BeanNameGenerator> nameGenerator() default
BeanNameGenerator.class;
```

在 `@SpringBootApplication` 注解中，`nameGenerator` 属性用于指定一个自定义的 `BeanNameGenerator` 实现。这通常用于定义 Spring 容器中 Bean 的名称，特别是在组件扫描时

主要作用：

- **自定义 Bean 名称：**通过实现 `BeanNameGenerator` 接口，你可以控制 Spring 为你的组件生成的 Bean 名称。例如，默认情况下，Spring 使用类名的小写形式作为 Bean 名称。使用自定义生成器，你可以改变这一行为。
- **冲突处理：**在一些复杂的应用中，可能会有多个相同名称的 Bean。在这种情况下，自定义名称生成器可以帮助解决冲突，确保每个 Bean 都有一个唯一的名称。
- **更好的可读性：**你可以为 Bean 生成更具描述性的名称，使得在调试和日志记录时更加直观。

使用示例：

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.beans.factory.config.BeanDefinitionRegistry;
import
org.springframework.beans.factory.support.BeanDefinitionRegistryPostProcessor;
import org.springframework.beans.factory.config.BeanDefinition;

@SpringBootApplication(nameGenerator = CustomBeanNameGenerator.class)
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```



```

}

public class CustomBeanNameGenerator implements BeanNameGenerator {
    @Override
    public String generateBeanName(BeanDefinition definition,
    BeanDefinitionRegistry registry) {
        // 自定义生成 Bean 名称的逻辑
        return "custom_" + definition.getBeanClassName();
    }
}

```

这个例子中，所有通过组件扫描注册的 Bean 都会以 `custom_` 前缀开头。

```

/**
 * Specify whether {@link Bean @Bean} methods should get proxied in order to
 * enforce
 * bean lifecycle behavior, e.g. to return shared singleton bean instances
 * even in
 * case of direct {@code @Bean} method calls in user code. This feature
 * requires
 * method interception, implemented through a runtime-generated CGLIB
 * subclass which
 * comes with limitations such as the configuration class and its methods not
 * being
 * allowed to declare {@code final}.
 * <p>
 * The default is {@code true}, allowing for 'inter-bean references' within
 * the
 * configuration class as well as for external calls to this configuration's
 * {@code @Bean} methods, e.g. from another configuration class. If this is
 * not needed
 * since each of this particular configuration's {@code @Bean} methods is
 * self-contained and designed as a plain factory method for container use,
 * switch
 * this flag to {@code false} in order to avoid CGLIB subclass processing.
 * <p>
 * Turning off bean method interception effectively processes {@code @Bean}
 * methods
 * individually like when declared on non-{@code @Configuration} classes,
 * a.k.a.
 * "Bean Lite Mode" (see {@link Bean @Bean's javadoc}). It is therefore
 * behaviorally
 * equivalent to removing the {@code @Configuration} stereotype.
 * @since 2.2
 * @return whether to proxy {@code @Bean} methods
 */
@AliasFor(annotation = Configuration.class)
boolean proxyBeanMethods() default true;

```

- proxyBeanMethods

指定@Bean方法是否应该被代理，以便强制执行bean生命周期行为，例如，即使在用户代码中直接调用@Bean方法的情况下，也要返回共享的单例bean实例。此特性需要方法拦截，通过运行时生成的CGLIB子类实现，该子类具有限制，例如配置类及其方法不允许声明final。

默认值为true，允许配置类中的“bean间引用”以及对该配置的@Bean方法的外部调用，例如来自另一个配置类。如果不需要这样做，因为每个特定配置的@Bean方法都是自包含的，并且设计为容器使用的普通工厂方法，请将此标志切换为false，以避免CGLIB子类处理。

关闭bean方法拦截可以有效地单独处理@Bean方法，就像在non-@Configuration类上声明时一样。

“@Bean精简模式”(参见@Bean的javadoc)。因此，它在行为上等同于移除@Configuration构造型。

返回:

是否代理@Bean方法

```
/**
 * Specify whether {@link Bean @Bean} methods should get proxied in order to
 * enforce
 * bean lifecycle behavior, e.g. to return shared singleton bean instances
 * even in
 * case of direct {@code @Bean} method calls in user code. This feature
 * requires
 * method interception, implemented through a runtime-generated CGLIB
 * subclass which
 * comes with limitations such as the configuration class and its methods not
 * being
 * allowed to declare {@code final}.
 * <p>
 * The default is {@code true}, allowing for 'inter-bean references' within
 * the
 * configuration class as well as for external calls to this configuration's
 * {@code @Bean} methods, e.g. from another configuration class. If this is
 * not needed
 * since each of this particular configuration's {@code @Bean} methods is
 * self-contained and designed as a plain factory method for container use,
 * switch
 * this flag to {@code false} in order to avoid CGLIB subclass processing.
 * <p>
 * Turning off bean method interception effectively processes {@code @Bean}
 * methods
 * individually like when declared on non-{@code @Configuration} classes,
 * a.k.a.
 * "@Bean Lite Mode" (see {@link Bean @Bean's javadoc}). It is therefore
 * behaviorally
 * equivalent to removing the {@code @Configuration} stereotype.
 * @since 2.2
 * @return whether to proxy {@code @Bean} methods
 */
@AliasFor(annotation = Configuration.class)
boolean proxyBeanMethods() default true;
```

@SpringBootConfiguration

本质上就是@Configuration注解，声明这个类是一个配置类。

注解说明：

指示一个类提供Spring Boot应用程序@Configuration。可以用作Spring的标准@Configuration注释的替代，以便可以自动找到配置(例如在测试中)。

应用程序应该只包含一个@SpringBootConfiguration，大多数习惯的Spring引导应用程序将从@SpringBootApplication继承它。

```
/**
 * Indicates that a class provides Spring Boot application
 * {@link Configuration @Configuration}. Can be used as an alternative to the
Spring's
 * standard {@code @Configuration} annotation so that configuration can be found
 * automatically (for example in tests).
 * <p>
 * Application should only ever include <em>one</em> {@code
@SpringBootConfiguration} and
 * most idiomatic Spring Boot applications will inherit it from
 * {@code @SpringBootApplication}.
 *
 * @author Phillip Webb
 * @author Andy Wilkinson
 * @since 1.4.0
 */
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Configuration
@Indexed
public @interface SpringBootConfiguration {

    /**
     * Specify whether {@link Bean @Bean} methods should get proxied in order to
enforce
     * bean lifecycle behavior, e.g. to return shared singleton bean instances
even in
     * case of direct {@code @Bean} method calls in user code. This feature
requires
     * method interception, implemented through a runtime-generated CGLIB
subclass which
     * comes with limitations such as the configuration class and its methods not
being
     * allowed to declare {@code final}.
     * <p>
     * The default is {@code true}, allowing for 'inter-bean references' within
the
     * configuration class as well as for external calls to this configuration's
     * {@code @Bean} methods, e.g. from another configuration class. If this is
not needed
     * since each of this particular configuration's {@code @Bean} methods is
     * self-contained and designed as a plain factory method for container use,
switch
     * this flag to {@code false} in order to avoid CGLIB subclass processing.

```

```

    * <p>
    * Turning off bean method interception effectively processes {@code @Bean}
methods
    * individually like when declared on non-{@code @Configuration} classes,
a.k.a.
    * "@Bean Lite Mode" (see {@link Bean @Bean's javadoc}). It is therefore
behaviorally
    * equivalent to removing the {@code @Configuration} stereotype.
    * @return whether to proxy {@code @Bean} methods
    * @since 2.2
    */
@AliasFor(annotation = Configuration.class)
boolean proxyBeanMethods() default true;
}

```

本质是一个@Configuration注解

```

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Configuration {
    @AliasFor(
        annotation = Component.class
    )
    String value() default "";

    boolean proxyBeanMethods() default true;
}

```

属性说明：

- proxyBeanMethods

指定@Bean方法是否应该被代理，以便强制执行bean生命周期行为，例如，即使在用户代码中直接调用@Bean方法的情况下，也要返回共享的单例bean实例。此特性需要方法拦截，通过运行时生成的CGLIB子类实现，该子类具有限制，例如配置类及其方法不允许声明final。

默认值为true，允许配置类中的“bean间引用”以及对该配置的@Bean方法的外部调用，例如来自另一个配置类。如果不需要这样做，因为每个特定配置的@Bean方法都是自包含的，并且设计为容器使用的普通工厂方法，请将此标志切换为false，以避免CGLIB子类处理。

关闭bean方法拦截可以有效地单独处理@Bean方法，就像在non-@Configuration类上声明时一样。

“@Bean精简模式”(参见@Bean的javadoc)。因此，它在行为上等同于移除@Configuration构造型。

返回：

是否代理@Bean方法

```

/**
 * Specify whether {@link Bean @Bean} methods should get proxied in order to
enforce
 * bean lifecycle behavior, e.g. to return shared singleton bean instances even
in
 * case of direct {@code @Bean} method calls in user code. This feature requires

```

```

* method interception, implemented through a runtime-generated CGLIB subclass
which
* comes with limitations such as the configuration class and its methods not
being
* allowed to declare {@code final}.
* <p>
* The default is {@code true}, allowing for 'inter-bean references' within the
* configuration class as well as for external calls to this configuration's
* {@code @Bean} methods, e.g. from another configuration class. If this is not
needed
* since each of this particular configuration's {@code @Bean} methods is
* self-contained and designed as a plain factory method for container use,
switch
* this flag to {@code false} in order to avoid CGLIB subclass processing.
* <p>
* Turning off bean method interception effectively processes {@code @Bean}
methods
* individually like when declared on non-{@code @Configuration} classes, a.k.a.
* "@Bean Lite Mode" (see {@link Bean @Bean's javadoc}). It is therefore
behaviorally
* equivalent to removing the {@code @Configuration} stereotype.
* @return whether to proxy {@code @Bean} methods
* @since 2.2
*/
@AliasFor(annotation = Configuration.class)
boolean proxyBeanMethods() default true;

```

@ComponentScan注解

1. 参考文章

- <https://juejin.cn/post/7092747462959448094>
- <https://www.cnblogs.com/jiafa/p/13806622.html>
- <https://www.cnblogs.com/heliusKing/p/13874172.html>

2.说明:

@ComponentScan主要用于将指定包路径下的、带有特定注解的对象自动装配到Spring容器中。（特定注解：@Component外，还有@Controller、@Repository、@Service也标注了@Component注解）
简单来讲：在Spring中通过定义bean的注解定义了一些bean，而Spring并不知道除非你告诉它去可以找到它们，而@ComponentScan的作用就是告诉Spring去哪个路径下可以找到这些bean。之前在xml配置bean的时候也会有扫描的相关配置（和xml方式的[context:component-scan](#)作用相似）。

3. 具体分析

源码

```

/**
 * Configures component scanning directives for use with {@link Configuration}
 classes.
 * Provides support parallel with Spring XML's {@code <context:component-scan>}
 element.

```

```

*
* <p>Either {@link #basePackageClasses} or {@link #basePackages} (or its alias
* {@link #value}) may be specified to define specific packages to scan. If
specific
* packages are not defined, scanning will occur from the package of the
* class that declares this annotation.
*
* <p>Note that the {@code <context:component-scan>} element has an
* {@code annotation-config} attribute; however, this annotation does not. This
is because
* in almost all cases when using {@code @ComponentScan}, default annotation
config
* processing (e.g. processing {@code @Autowired} and friends) is assumed.
Furthermore,
* when using {@link AnnotationConfigApplicationContext}, annotation config
processors are
* always registered, meaning that any attempt to disable them at the
* {@code @ComponentScan} level would be ignored.
*
* <p>See {@link Configuration @Configuration}'s Javadoc for usage examples.
*
* @author Chris Beams
* @author Juergen Hoeller
* @author Sam Brannen
* @since 3.1
* @see Configuration
*/
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Documented
@Repeatable(ComponentScans.class)
public @interface ComponentScan {

    /**
     * 定义待扫描的包路径名
     */
    * Alias for {@link #basePackages}.
    * <p>Allows for more concise annotation declarations if no other attributes
    * are needed &mdash; for example, {@code @ComponentScan("org.my.pkg")}
    * instead of {@code @ComponentScan(basePackages = "org.my.pkg")}.
    */
    @AliasFor("basePackages")
    String[] value() default {};

    /**
     * 定义待扫描的包路径名
     */
    * Base packages to scan for annotated components.
    * <p>{@link #value} is an alias for (and mutually exclusive with) this
    * attribute.
    * <p>Use {@link #basePackageClasses} for a type-safe alternative to
    * String-based package names.
    */
    @AliasFor("value")
    String[] basePackages() default {};

    /**
     * 定义待扫描的类名
     */

```

```

    * Type-safe alternative to {@link #basePackages} for specifying the packages
    * to scan for annotated components. The package of each class specified will
    be scanned.
    * <p>Consider creating a special no-op marker class or interface in each
    package
    * that serves no purpose other than being referenced by this attribute.
    */
    Class<?>[] basePackageClasses() default {};

    // 自定义bean名称生成器
    /**
    * The {@link BeanNameGenerator} class to be used for naming detected
    components
    * within the Spring container.
    * <p>The default value of the {@link BeanNameGenerator} interface itself
    indicates
    * that the scanner used to process this {@code @ComponentScan} annotation
    should
    * use its inherited bean name generator, e.g. the default
    * {@link AnnotationBeanNameGenerator} or any custom instance supplied to the
    * application context at bootstrap time.
    * @see
    AnnotationConfigApplicationContext#setBeanNameGenerator(BeanNameGenerator)
    * @see AnnotationBeanNameGenerator
    * @see FullyQualifiedAnnotationBeanNameGenerator
    */
    Class<? extends BeanNameGenerator> nameGenerator() default
    BeanNameGenerator.class;

    //
    /**
    * The {@link ScopeMetadataResolver} to be used for resolving the scope of
    detected components.
    */
    Class<? extends ScopeMetadataResolver> scopeResolver() default
    AnnotationScopeMetadataResolver.class;

    //
    /**
    * Indicates whether proxies should be generated for detected components,
    which may be
    * necessary when using scopes in a proxy-style fashion.
    * <p>The default is defer to the default behavior of the component scanner
    used to
    * execute the actual scan.
    * <p>Note that setting this attribute overrides any value set for {@link
    #scopeResolver}.
    * @see ClassPathBeanDefinitionScanner#setScopedProxyMode(ScopedProxyMode)
    */
    ScopedProxyMode scopedProxy() default ScopedProxyMode.DEFAULT;

    // 需要扫描包中的那些资源，默认是： **/*.class，即会扫描指定包中所有的class文件
    /**
    * Controls the class files eligible for component detection.
    * <p>Consider use of {@link #includeFilters} and {@link #excludeFilters}
    * for a more flexible approach.

```

```

    */
    String resourcePattern() default
ClassPathScanningCandidateComponentProvider.DEFAULT_RESOURCE_PATTERN;

    // 对扫描的类是否启用默认过滤器，默认为true
    /**
     * Indicates whether automatic detection of classes annotated with {@code
@Component}
     * {@code @Repository}, {@code @Service}, or {@code @Controller} should be
enabled.
     */
    boolean useDefaultFilters() default true;

    // 满足过滤器条件的，才能被扫描
    /**
     * Specifies which types are eligible for component scanning.
     * <p>Further narrows the set of candidate components from everything in
{@link #basePackages}
     * to everything in the base packages that matches the given filter or
filters.
     * <p>Note that these filters will be applied in addition to the default
filters, if specified.
     * Any type under the specified base packages which matches a given filter
will be included,
     * even if it does not match the default filters (i.e. is not annotated with
{@code @Component}).
     * @see #resourcePattern()
     * @see #useDefaultFilters()
     */
    Filter[] includeFilters() default {};

    // 满足过滤器条件的，不会被扫描
    /**
     * Specifies which types are not eligible for component scanning.
     * @see #resourcePattern
     */
    Filter[] excludeFilters() default {};

    // 是否延迟初始化被注册的bean
    /**
     * Specify whether scanned beans should be registered for lazy
initialization.
     * <p>Default is {@code false}; switch this to {@code true} when desired.
     * @since 4.1
     */
    boolean lazyInit() default false;

    /**
     * Declares the type filter to be used as an {@linkplain
ComponentScan#includeFilters
     * include filter} or {@linkplain ComponentScan#excludeFilters exclude
filter}.
     */
    @Retention(RetentionPolicy.RUNTIME)
    @Target({})

```



```

@interface Filter {

    /**
     * The type of filter to use.
     * <p>Default is {@link FilterType#ANNOTATION}.
     * @see #classes
     * @see #pattern
     */
    FilterType type() default FilterType.ANNOTATION;

    /**
     * Alias for {@link #classes}.
     * @see #classes
     */
    @AliasFor("classes")
    Class<?>[] value() default {};

    /**
     * The class or classes to use as the filter.
     * <p>The following table explains how the classes will be interpreted
     * based on the configured value of the {@link #type} attribute.
     * <table border="1">
     * <tr><th>{@code FilterType}</th><th>Class Interpreted As</th></tr>
     * <tr><td>{@link FilterType#ANNOTATION ANNOTATION}</td>
     * <td>the annotation itself</td></tr>
     * <tr><td>{@link FilterType#ASSIGNABLE_TYPE ASSIGNABLE_TYPE}</td>
     * <td>the type that detected components should be assignable to</td>
    </tr>
     * <tr><td>{@link FilterType#CUSTOM CUSTOM}</td>
     * <td>an implementation of {@link TypeFilter}</td></tr>
     * </table>
     * <p>When multiple classes are specified, <em>OR</em> logic is applied
     * &mdash; for example, "include types annotated with {@code @Foo} OR
    {@code @Bar}".
     * <p>Custom {@link TypeFilter TypeFilters} may optionally implement any
    of the
     * following {@link org.springframework.beans.factory.Aware Aware}
    interfaces, and
     * their respective methods will be called prior to {@link
    TypeFilter#match match}:
     * <ul>
     * <li>{@link org.springframework.context.EnvironmentAware
    EnvironmentAware}</li>
     * <li>{@link org.springframework.beans.factory.BeanFactoryAware
    BeanFactoryAware}</li>
     * <li>{@link org.springframework.beans.factory.BeanClassLoaderAware
    BeanClassLoaderAware}</li>
     * <li>{@link org.springframework.context.ResourceLoaderAware
    ResourceLoaderAware}</li>
     * </ul>
     * <p>Specifying zero classes is permitted but will have no effect on
    component
     * scanning.
     * @since 4.2
     * @see #value
     * @see #type

```

```

        */
        @AliasFor("value")
        Class<?>[] classes() default {};

        /**
         * The pattern (or patterns) to use for the filter, as an alternative
         * to specifying a Class {@link #value}.
         * <p>If {@link #type} is set to {@link FilterType#ASPECTJ ASPECTJ},
         * this is an AspectJ type pattern expression. If {@link #type} is
         * set to {@link FilterType#REGEX REGEX}, this is a regex pattern
         * for the fully-qualified class names to match.
         * @see #type
         * @see #classes
         */
        String[] pattern() default {};
    }
}

```

Repeatable注解

```
@Repeatable(ComponentScans.class)
```

源码

```

/**
 * The annotation type {@code java.lang.annotation.Repeatable} is
 * used to indicate that the annotation type whose declaration it
 * (meta-)annotates is <em>repeatable</em>. The value of
 * {@code @Repeatable} indicates the <em>containing annotation
 * type</em> for the repeatable annotation type.
 *
 * @since 1.8
 * @jls 9.6 Annotation Types
 * @jls 9.7 Annotations
 */
@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.ANNOTATION_TYPE)
public @interface Repeatable {
    /**
     * Indicates the <em>containing annotation type</em> for the
     * repeatable annotation type.
     * @return the containing annotation type
     */
    Class<? extends Annotation> value();
}

```

用于指示它(元)注释的声明的注释类型是可重复的。@Repeatable的值表示可重复标注类型的包含标注类型。

相关文章参考: <https://cloud.tencent.com/developer/article/1579167>

scopedProxy注解

<https://www.cnblogs.com/jiafa/p/13806622.html>

scopeResolver注解

<https://www.cnblogs.com/heliusKing/p/13874172.html>

Spring中的注解说明

用于指定需要扫描的组件内容

```
@ComponentScan(excludeFilters = { @Filter(type = FilterType.CUSTOM, classes =  
    TypeExcludeFilter.class),  
    @Filter(type = FilterType.CUSTOM, classes =  
    AutoConfigurationExcludeFilter.class) })
```

@ComponentScan 注解中的 excludeFilters 属性用于指定在组件扫描时要排除的特定类型的组件。

使用了两个自定义的过滤器: TypeExcludeFilter 和 AutoConfigurationExcludeFilter。下面是它们的作用:

- TypeExcludeFilter

这个过滤器用于排除特定类型的类, 通常用于控制哪些组件不会被 Spring 扫描和注册。例如, 可以用于排除某些实现了特定接口的类

这段代码定义了 TypeExcludeFilter 类, 它实现了 TypeFilter 接口和 BeanFactoryAware 接口。其主要功能是根据条件过滤组件, 以控制哪些类被扫描。

setBeanFactory: 设置 BeanFactory 实例, 用于获取 Spring 上下文中的 Beans。

match 方法: 检查当前类是否与任何代理类匹配, 返回 true 表示匹配, false 表示不匹配。它首先确保 beanFactory 是 ListableBeanFactory, 然后遍历所有代理过滤器进行匹配。

getDelegates: 获取所有类型为 TypeExcludeFilter 的 Bean, 避免重复查询。

equals 和 hashCode 方法: 这两个方法抛出异常, 表示该过滤器不支持比较。

你想深入了解这个类的某个方面吗?

- AutoConfigurationExcludeFilter

这个过滤器专门用于排除自动配置类。Spring Boot 的自动配置机制会根据类路径和其他条件自动配置 Beans。有时, 你可能希望排除某些自动配置类, 以便手动配置或替换它们。

`AutoConfigurationExcludeFilter` 是一个 `TypeFilter` 的实现，用于匹配注册的自动配置类。以下是其关键功能：

`setBeanClassLoader`：设置用于加载类的 `ClassLoader`。

`match` 方法：判断给定的类是否是自动配置类，调用 `isConfiguration` 和 `isAutoConfiguration` 方法。

`isConfiguration`：检查类是否被 `@Configuration` 注解标记。

`isAutoConfiguration`：判断类是否被 `@AutoConfiguration` 注解标记，或者是否在自动配置类列表中。

`getAutoConfigurations`：加载所有自动配置类，使用 `SpringFactoriesLoader` 和 `ImportCandidates`

@Filter注解

```
@Retention(RetentionPolicy.RUNTIME)
@Target({})
public @interface Filter {
    FilterType type() default FilterType.ANNOTATION;

    @AliasFor("classes")
    Class<?>[] value() default {};

    @AliasFor("value")
    Class<?>[] classes() default {};

    String[] pattern() default {};
}
```

@EnableAutoConfiguration

1.参考文章

- 关于@EnableAutoConfiguration中的@import: <https://www.cnblogs.com/kevin-yuan/p/13583269.html>
- @Import注解: <https://juejin.cn/post/6844904035212853255>
- @EnableAutoConfiguration注解详解: <https://juejin.cn/post/7202573260658868283>

2.详解

@EnableAutoConfiguration中会引入AutoConfigurationImportSelector，其主要就是做了如下事情

```
1. AutoConfigurationImportSelector#getCandidateConfigurations
    1.1 AutoConfigurationImportSelector#getSpringFactoriesLoaderFactoryClass() //
    返回 org.springframework.boot.autoconfigure.EnableAutoConfiguration
    1.2 SpringFactoriesLoader#loadFactoryNames // 通过 SpringBoot SPI 去加载
    spring.factories 中
    key=org.springframework.boot.autoconfigure.EnableAutoConfiguration 的值
```

通过 `AutoConfigurationImportSelector#selectImports()` 返回的自动配置类，
`ConfigurationClassParser#processImports()` 方法再去加载和处理这些配置类。这就是
`@EnableAutoConfiguration` 的核心原理。

2. WebApplicationType:

- **NONE**: 应用程序不应作为网络应用程序运行，并且不应启动嵌入式网络服务器。
- **SERVLET**: 应用程序应作为基于 Servlet 的网络应用程序运行，并启动嵌入式 Servlet 网络服务器。
- **REACTIVE**: 应用程序应作为响应式网络应用程序运行，并启动嵌入式响应式网络服务器。

静态方法，用于根据类路径中的类来推断应用程序的类型。

- 它首先检查是否存在响应式 Web 应用的相关类（如 `DispatcherHandler`），并确保不同时存在传统的 Web MVC 类（如 `DispatcherServlet`）或 Jersey 类（如 `ServletContainer`）。如果仅存在响应式类，则返回 `REACTIVE`。
- 如果存在所有 Servlet 指标类（如 `Servlet`），则返回 `SERVLET`。
- 如果上述类都不存在，则返回 `NONE`。

```
public enum WebApplicationType {  
  
    /**  
     * The application should not run as a web application and should not start  
an  
     * embedded web server.  
     */  
    NONE,  
  
    /**  
     * The application should run as a servlet-based web application and should  
start an  
     * embedded servlet web server.  
     */  
    SERVLET,  
  
    /**  
     * The application should run as a reactive web application and should start  
an  
     * embedded reactive web server.  
     */  
    REACTIVE;  
  
    private static final String[] SERVLET_INDICATOR_CLASSES = {  
"javax.servlet.Servlet",  
        "org.springframework.web.context.ConfigurableWebApplicationContext"  
    };  
};
```

```

        private static final String WEBMVC_INDICATOR_CLASS =
"org.springframework.web.servlet.DispatcherServlet";

        private static final String WEBFLUX_INDICATOR_CLASS =
"org.springframework.web.reactive.DispatcherHandler";

        private static final String JERSEY_INDICATOR_CLASS =
"org.glassfish.jersey.servlet.ServletContainer";

        static WebApplicationType deduceFromClasspath() {
            if (ClassUtils.isPresent(WEBFLUX_INDICATOR_CLASS, null) &&
!ClassUtils.isPresent(WEBMVC_INDICATOR_CLASS, null)
                && !ClassUtils.isPresent(JERSEY_INDICATOR_CLASS, null)) {
                return WebApplicationType.REACTIVE;
            }
            for (String className : SERVLET_INDICATOR_CLASSES) {
                if (!ClassUtils.isPresent(className, null)) {
                    return WebApplicationType.NONE;
                }
            }
            return WebApplicationType.SERVLET;
        }
    }
}

```

2. getSpringFactoriesInstances

三个调用getSpringFactoriesInstances方法来初始化SpringApplication的内部类

```

this.bootstrapRegistryInitializers = new ArrayList<> (
    getSpringFactoriesInstances(BootstrapRegistryInitializer.class));
    setInitializers((Collection)
getSpringFactoriesInstances(ApplicationContextInitializer.class));
    setListeners((Collection)
getSpringFactoriesInstances(ApplicationListener.class));

```